

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Лабораторная работа №4
по дисциплине
«Методы тестирования программного обеспечения»
Вариант 2

Студент,

группы 5130201/30101

_____ Мелешенко С. И.

Преподаватель

_____ Курочкин М. А.

« _____ » _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	4
1 Постановка задачи	5
2 Средства автоматизации тестирования	6
2.1 JUnit	6
2.1.1 Основные особенности JUnit	6
2.1.2 Преимущества использования JUnit	6
2.1.3 Функционал библиотеки	7
2.1.4 Этапы написания тестов	7
2.2 Selenium WebDriver	8
2.3 Page Object	9
2.3.1 Основные принципы Page Object	9
2.4 TestNG	10
2.4.1 Основные особенности TestNG	10
2.4.2 Основные аннотации TestNG	10
2.4.3 Этапы написания тестов в TestNG	10
3 Задача 1. Юнит-тестирование библиотеки на Java (JUnit)	12
3.1 Тестирование площади круга (CircleAreaTest.java)	12
3.2 Тестирование длины окружности (CirclePerimeterTest.java)	13
3.3 Тестирование площади прямоугольника (RectangleArea- Test.java)	14
3.4 Тестирование периметра прямоугольника (RectanglePerimeter- Test.java)	15
3.5 Результаты тестирования	15
4 Задача 2. UI-тестирование веб-сайта (Selenium WebDriver + TestNG)	18
4.1 Настройка драйвера	18
4.2 NavigationTraining1	19
4.2.1 Метод навигации	19
4.2.2 Метод отладки	19
4.2.3 Основной тест	19
4.3 HomePageNavigationTests	20
4.3.1 Тест 1: Проверка заголовка страницы	20
4.3.2 Тест 2: Проверка количества пунктов меню	21
4.3.3 Тест 3: Проверка текстов пунктов меню	21
4.3.4 Тест 4: Проверка ссылок пунктов меню	22

4.3.5	Тест 5: Проверка кликабельности пунктов меню	22
4.3.6	Тест 6: Проверка имени залогиненного пользователя	23
4.3.7	Тест 7: Проверка аватара пользователя	23
4.3.8	Тест 8: Проверка логотипа сайта	24
4.3.9	Тест 9: Проверка футера страницы	25
4.3.10	Тест 10: Проверка responsive-классов	25
4.4	Реализация тестов	25
4.5	Результаты тестирования	26
Заключение		28
А Приложения		31
A.1	Листинг 1 — CircleAreaTest.java	31
A.2	Листинг 2 — RectangleAreaTest.java	32
A.3	Листинг 3 — CirclePerimeterTest.java	33
A.4	Листинг 4 — RectanglePerimeterTest.java	35
A.5	Листинг 5 — NavigationTraining1.java	36
A.6	Листинг 6 — HomePageNavigationTests.java	39

Введение

Автоматизированное тестирование — это метод проверки программного обеспечения с использованием специальных инструментов и скриптов. В отличие от ручного тестирования, тестовые примеры разрабатываются однократно и многократно выполняются автоматически, а полученные результаты сравниваются с ожидаемыми.

Автоматизация тестирования значительно ускоряет процесс проверки приложений, минимизирует человеческий фактор и позволяет создавать стандартизированные тесты. Однако автоматизация не может полностью заменить ручное тестирование, особенно в оценке удобства интерфейса и выявления неожиданных дефектов.

В данной работе изучаются инструменты автоматизированного тестирования: фреймворк JUnit для модульного тестирования, а также Selenium WebDriver и TestNG для UI-тестирования веб-приложений.

1 Постановка задачи

Цель работы — изучение и применение инструментов автоматизированного тестирования: фреймворка JUnit для модульного-тестирования и Selenium WebDriver вместе с TestNG для UI-тестирования веб-приложений.

Задачи:

1. Создать юнит-тесты для библиотеки lab-1.0.jar с использованием фреймворка JUnit:

- Разработать юнит-тесты для четырех методов класса ArrayUtils.java.
- Запустить тесты через используемую среду разработки.
- Продемонстрировать результаты выполнения тестов.

2. Провести тестирование веб-сайта с использованием Selenium WebDriver и TestNG:

- Написать два теста, проверяющих корректное отображение и работу страниц веб-сайта.
- Организовать код тестов в соответствии с Java Code Convention.
- Запустить тесты через конфигурационный файл TestNG suite.xml.

2 Средства автоматизации тестирования

2.1 JUnit

JUnit — это фреймворк, созданный для тестирования программного обеспечения на языке Java. Он предназначен для разработки и выполнения автоматизированных тестов, что дает разработчикам возможность проверять правильность функционирования своего кода и обнаруживать ошибки на ранних стадиях разработки.

2.1.1 Основные особенности JUnit

- **Модульное тестирование:** JUnit поддерживает тестирование отдельных модулей, позволяя разработчикам проверять работу отдельных компонентов (методов и классов) независимо друг от друга.
- **Аннотации:** JUnit использует аннотации, такие как `@Test`, `@Before`, `@After`, которые упрощают написание тестов и делают код более читабельным. Например, аннотация `@Before` позволяет выполнять определенный код перед каждым тестом, а `@After` — после его завершения.
- **Ассерты:** JUnit предоставляет различные методы ассертов, такие как `assertEquals`, `assertTrue`, `assertNotNull`, которые помогают проверять ожидаемые результаты тестов.
- **Группировка тестов:** Тесты могут быть сгруппированы с помощью аннотации `@Suite`, что позволяет запускать их вместе.
- **Интеграция:** JUnit просто интегрируется с различными инструментами и фреймворками, такими как Maven, Gradle и широко используемые IDE, включая Eclipse и IntelliJ IDEA, что упрощает процесс тестирования.

2.1.2 Преимущества использования JUnit

- **Упрощение процесса тестирования:** JUnit делает написание тестов более удобным и структурированным.
- **Автоматизация:** Позволяет автоматизировать тесты, что снижает вероятность ошибок и повышает качество кода.
- **Поддержка непрерывной интеграции:** JUnit хорошо совместим с системами непрерывной интеграции, позволяя запускать тесты автоматически при каждом изменении кода.
- **Снижение затрат на отладку:** Регулярное тестирование помогает выявлять проблемы на ранних стадиях, что уменьшает расходы на их исправление.

2.1.3 Функционал библиотеки

JUnit имеет широкий функционал для проверки совпадения ожидаемого результата, и результата полученного тестируемым методом.

Класс `junit.framework.Assert` предоставляет набор статических методов для проверки различных условий в тестах.

- `assertEquals(expected, actual)` - проверяет равенство двух значений. Если значения не равны, тест завершается с ошибкой. Имеет перегрузки для различных типов данных.
- `assertFalse(condition)` - проверяет, что переданное булево значение является `false`. Если значение `true`, тест завершается с ошибкой.
- `assertNotNull(object)` - проверяет, что объект не является `null`. Если объект `null`, тест завершается с ошибкой.
- `assertNull(object)` - проверяет, что объект является `null`. Если объект не `null`, тест завершается с ошибкой.
- `assertNotSame(unexpected, actual)` - проверяет, что два объекта не ссылаются на один и тот же экземпляр. Если ссылки идентичны, тест завершается с ошибкой.
- `assertSame(expected, actual)` - проверяет, что два объекта ссылаются на один и тот же экземпляр. Если ссылки разные, тест завершается с ошибкой.
- `assertTrue(condition)` - проверяет, что переданное булево значение является `true`. Если значение `false`, тест завершается с ошибкой.

Класс `junit.framework.TestCase` наследуется от `junit.framework.Assert` и предоставляет базовую функциональность для создания тестовых случаев.

- `run()` - основной метод, выполняющий тест. Содержит логику запуска и выполнения тестового случая.
- `setUp()` - метод, выполняемый перед каждым тестом. Используется для инициализации тестового окружения и подготовки данных. В аннотационной версии JUnit 4+ заменён на `@Before`.
- `tearDown()` - метод, выполняемый после каждого теста. Используется для очистки ресурсов после выполнения теста. В аннотационной версии JUnit 4+ заменён на `@After`.

2.1.4 Этапы написания тестов

1. **Реализация теста:** Написание тестового метода и аннотирование его с помощью. `@Test`

2. **Настройка и очистка** Использование аннотаций `@Before` и `@After` для выполнения операций перед и после теста.
3. **Запуск тестов:** Использование встроенных средств IDE или командной строки для выполнения тестов.

2.2 Selenium WebDriver

Selenium WebDriver — это фреймворк с открытым исходным кодом для автоматизации тестирования веб-приложений. Он предоставляет программный интерфейс для взаимодействия с браузерами, позволяя эмулировать действия пользователя на веб-страницах. **Основные особенности Selenium WebDriver:**

- **Кроссбраузерность:** Поддержка всех популярных браузеров, включая Chrome, Firefox, Safari, Edge и Opera.
- **Многоязычность:** Возможность написания тестовых скриптов на различных языках программирования — Java, Python, C#, Ruby, JavaScript, PHP и Perl.
- **Прямое взаимодействие:** WebDriver напрямую отправляет команды браузеру и получает результаты, что обеспечивает более точное воспроизведение пользовательских действий.
- **Кроссплатформенность:** Возможность запуска тестов на различных операционных системах (Windows, MacOS, Linux).
- **Параллельное выполнение:** Поддержка одновременного запуска тестов в разных браузерах для ускорения тестирования.

Архитектура Selenium WebDriver состоит из четырех основных компонентов:

- **Selenium Client Libraries:** Набор библиотек для различных языков программирования, позволяющих писать и запускать тесты на предпочитаемом языке. JSON Wire Protocol: REST API на основе JSON, обеспечивающий передачу информации между клиентом и сервером через HTTP.
- **Browser Drivers:** Специфичные для каждого браузера драйверы (ChromeDriver, GeckoDriver для Firefox и др.), которые получают команды и выполняют их в соответствующем браузере.
- **Browsers:** Сами браузеры, в которых выполняются тестовые сценарии.

Этапы работы с WebDriver:

1. **Инициализация:** Создание экземпляра WebDriver и открытие браузера.
2. **Навигация:** Переход к нужной веб-странице с помощью метода `get()`.
3. **Поиск элементов:** Обнаружение элементов на странице с использованием различных локаторов (ID, XPath, CSS-селекторы и др.).

4. **Взаимодействие:** Выполнение действий над элементами (клик, ввод текста, выбор из выпадающих списков и т.д.).
5. **Ожидание:** Использование явных и неявных ожиданий для синхронизации с динамическими элементами страницы.
6. **Проверка:** Получение информации о состоянии элементов и проверка результатов.
7. **Завершение:** Закрытие браузера и освобождение ресурсов.

Принцип выполнения команд:

1. Команда из тестового скрипта преобразуется в HTTP-запрос через JSON Wire Protocol.
2. Запрос передается соответствующему драйверу браузера.
3. Драйвер интерпретирует запрос и выполняет необходимые действия в браузере.
4. Результат действия возвращается обратно в виде HTTP-ответа.
5. Ответ преобразуется в формат, понятный тестовому скрипту.

2.3 Page Object

Page Object — шаблон проектирования для автоматизации тестирования веб-приложений. Он помогает инкапсулировать работу с отдельными элементами страницы, что позволяет использовать одни и те же объекты страниц в различных тестах.

2.3.1 Основные принципы Page Object

1. Одна страница — один класс. Каждая веб-страница представляется отдельным классом.
2. Локаторы хранятся внутри класса, методы описывают действия пользователя.
3. Тесты работают только через методы Page Object, не обращаясь напрямую к элементам страницы.

Основные преимущества использования паттерна Page Object:

- разделение кода тестов и кода описания страниц;
- объединение всех действий по работе с веб-страницей в одном месте.

Паттерн Page Object в Selenium реализован с помощью библиотеки PageFactory и класса страницы. Page Object представляет собой отдельный класс, содержащий локаторы элементов, методы для работы с ними и конструктор, принимающий в качестве параметра объект WebDriver. Методы класса Page Object могут возвращать объекты других Page Object классов.

2.4 TestNG

TestNG (Test Next Generation) — это современный фреймворк для тестирования Java-приложений, предлагающий более мощные возможности, чем JUnit. Он поддерживает различные виды тестирования: модульное, функциональное, интеграционное и End-to-end тестирование.

2.4.1 Основные особенности TestNG

- TestNG использует расширенную систему аннотаций для управления жизненным циклом тестов.
- Тесты можно настраивать с помощью аннотаций Java и XML-файлов конфигурации.
- Тесты можно объединять в группы с помощью аннотации `@Test(groups="groupName")`, что позволяет запускать только определённые группы тестов.
- Параллельное выполнение: TestNG поддерживает параллельное выполнение тестов на уровне методов, классов и тестовых наборов.
- Поддержка параметризованных тестов. Аннотации `@Parameters` и `@DataProvider` позволяют передавать параметры в тесты, что упрощает тестирование с различными входными данными.
- Встроенная поддержка многопоточности для нагрузочного тестирования.

2.4.2 Основные аннотации TestNG

- `@Test` — основная аннотация для тестовых методов;
- `@BeforeSuite` / `@AfterSuite` — выполняются до/после всего набора тестов;
- `@BeforeTest` / `@AfterTest` — выполняются до/после тестового блока в XML;
- `@BeforeClass` / `@AfterClass` — выполняются до/после методов класса;
- `@BeforeMethod` / `@AfterMethod` — выполняются до/после каждого тестового метода;
- `@BeforeGroups` / `@AfterGroups` — выполняются до/после группы тестов;
- `@Parameters` — для параметризации тестов через XML;
- `@DataProvider` — для подачи тестовых данных через код.

2.4.3 Этапы написания тестов в TestNG

1. Создание тестового класса, написание тестовых методов с аннотацией `@Test`.
2. Настройка пред- и пост-условий: использование аннотаций `@BeforeSuite`, `@AfterSuite`, `@BeforeTest`, `@AfterTest`, `@BeforeMethod`, `@AfterMethod` для настройки и очистки.
3. Конфигурация тестов: добавление информации о тесте (имя класса, группы, параметры) через XML-файл или аннотации.

4. Запуск тестов через IDE (IntelliJ IDEA, Eclipse), Maven (`mvn test`) или XML-конфигурацию для управления тестовыми наборами.

5. Анализ результатов: просмотр отчётов TestNG (HTML-отчёты), которые включают информацию о пройденных и неудачных тестах, времени выполнения и других метриках.

3 Задача 1. Юнит-тестирование библиотеки на Java (JUnit)

Дано

- Библиотечный файл lab-1.0.jar (в варианте пользователя — класс GeometryUtils с методами геометрических вычислений).
- Методы для тестирования (выбраны 4):
 - calculateCircleArea(double radius) – площадь круга.
 - calculateCirclePerimeter(double radius) – длина окружности.
 - calculateRectangleArea(double width, double height) – площадь прямоугольника.
 - calculateRectanglePerimeter(double width, double height) – периметр прямоугольника.

Настройка проекта 1. Создан Java-проект, подключена библиотека lab-1.0.jar и JUnit 4 (используется параметризованный раннер Parameterized.class).

2. Класс GeometryUtils инициализируется один раз перед всеми тестами с помощью @BeforeClass.

3. Все тесты используют параметризацию: входные данные и ожидаемые результаты задаются через метод @Parameters, возвращающий коллекцию массивов.

3.1 Тестирование площади круга (CircleAreaTest.java)

Параметризация

Метод data() возвращает набор объектов:

- {1.0, 3.141592653589793} – нормальный радиус.
- {0.5, 0.7853981633974483} – дробный радиус.
- {0.0, 0.0} – нулевой радиус (граничное значение).
- {Double.MAX_VALUE, Double.POSITIVE_INFINITY} – максимальный радиус (ожидается переполнение -> бесконечность).
- {-1.0, -1.0} – отрицательный радиус (ожидается исключение).

Логика теста

- Если radius <= 0 – проверяется выброс IllegalArgumentException.
- Если ожидаемый результат бесконечен – проверяется Double.isInfinite().
- Иначе – сравнение с погрешностью 0.0001.

Классы эквивалентности

Условие	Допустимые значения	Недопустимые значения
radius	>0	<=0

Граничные значения

- radius = 0 (переход от положительных к недопустимым)
- radius = Double.MAX_VALUE (максимальное представимое положительное число)
- radius = -1.0 (отрицательное)

Обнаруженный дефект

Метод calculateCircleArea реализован верно, дефектов не обнаружено.

3.2 Тестирование длины окружности (CirclePerimeterTest.java)

Параметризация

Метод data() возвращает набор объектов:

- {1.0, 6.283185307179586} – нормальный радиус (ожидается $2\pi \cdot 1$).
- {0.5, 3.141592653589793} – дробный радиус ($2\pi \cdot 0.5$).
- {0.0, 0.0} – нулевой радиус (граничное значение).
- {Double.MAX_VALUE, Double.POSITIVE_INFINITY} – максимальный радиус (ожидается переполнение -> бесконечность).
- {-1.0, -1.0} – отрицательный радиус (ожидается исключение).

Логика теста

- Если radius <= 0 – проверяется выброс IllegalArgumentException.
- Если ожидаемый результат бесконечен – проверяется Double.isInfinite().
- Иначе – сравнение с погрешностью 0.0001.

Классы эквивалентности

Условие	Допустимые значения	Недопустимые значения
radius	>0	<=0

Граничные значения

- radius = 0 (переход от положительных к недопустимым)
- radius = Double.MAX_VALUE (максимальное представимое положительное число)
- radius = -1.0 (отрицательное)

Обнаруженный дефект

Метод `calculateCirclePerimeter` возвращает $\pi \cdot r$ вместо $2 \cdot \pi \cdot r$ – отсутствует умножение на 2. Дефект проявляется для всех положительных радиусов.

3.3 Тестирование площади прямоугольника (RectangleAreaTest.java)

Параметризация

Метод `data()` возвращает набор объектов с четырьмя параметрами: `width`, `height`, `expected`, `shouldThrowException`:

- {2.0, 3.0, 6.0, false} – нормальные стороны.
- {0.5, 4.0, 2.0, false} – дробные стороны.
- {0.0, 5.0, 0.0, true} – нулевая ширина (ожидается исключение).
- {Double.MAX_VALUE, 1.0, Double.POSITIVE_INFINITY, false} – переполнение.
- {-1.0, 2.0, -1.0, true} – отрицательная сторона (ожидается исключение).

Логика теста

- Если `shouldThrowException == true` – проверяется выброс `IllegalArgumentException`.
- Если ожидаемый результат бесконечен – проверяется `Double.isInfinite()`.
- Иначе – сравнение с погрешностью 0.0001.

Классы эквивалентности

Условие	Допустимые значения	Недопустимые значения
width	>0	≤ 0
height	>0	≤ 0

Граничные значения

- `width = 0` или `height = 0` (переход от положительных к недопустимым)
- `width = Double.MAX_VALUE`, `height = 1` (проверка переполнения)
- `width = -1.0`, `height = 2.0` (отрицательные значения)

Обнаруженный дефект

Метод `calculateRectangleArea` не выбрасывает `IllegalArgumentException` при `width = 0`. При этом отрицательные стороны обрабатываются корректно. Дефект связан с проверкой нулевых значений.

3.4 Тестирование периметра прямоугольника (RectanglePerimeterTest.java)

Параметризация

Метод `data()` возвращает набор объектов (`width`, `height`, `expected`, `shouldThrowException`):

- {2.0, 3.0, 10.0, false} – нормальные стороны (периметр $2 \cdot (2+3)=10$).
- {0.5, 4.0, 9.0, false} – дробные стороны $2 \cdot (0.5+4)=9$.
- {0.0, 5.0, 10.0, true} – нулевая ширина (ожидается исключение).
- {Double.MAX_VALUE, 1.0, Double.POSITIVE_INFINITY, false} – переполнение.
- {-1.0, 2.0, -1.0, true} – отрицательная сторона (ожидается исключение).

Логика теста

Аналогична площади прямоугольника.

Классы эквивалентности

Условие	Допустимые значения	Недопустимые значения
width	>0	≤ 0
height	>0	≤ 0

Граничные значения

- width = 0 или height = 0
- width = Double.MAX_VALUE, height = 1 (переполнение)
- Отрицательные стороны

Обнаруженный дефект

Метод `calculateRectanglePerimeter` работает корректно: правильно считает периметр, обрабатывает недопустимые значения (выбрасывает исключения) и корректно возвращает бесконечность при переполнении.

3.5 Результаты тестирования

Разработаны параметризованные юнит-тесты для четырёх методов библиотеки `GeometryUtils` с использованием JUnit 4. Тесты покрывают классы эквивалентности (положительные, нулевые, отрицательные значения) и граничные условия (максимальные значения, переполнение).

Результаты выполнения тестов (подтверждены скриншотами):

Таблица 1

Результаты тестирования методов GeometryUtils

Метод	Всего тестов	Пройдено	Не пройдено	Дефект
calculateCircleArea	5	5	0	Нет
calculateCirclePerimeter	5	3	2	Ошибка формулы: возвращает $\pi \cdot r$ вместо $2\pi \cdot r$
calculateRectangleArea	5	4	1	Не выбрасывает исключение при $\text{width} = 0$ (возвращает 0)
calculateRectanglePerimeter	5	5	0	Нет

Итого: 20 тестов. Пройдено: 17. Не пройдено: 3.

Библиотека `GeometryUtils` содержит два дефекта. Все тесты написаны в соответствии с требованиями (параметризация, классы эквивалентности, граничные значения). Скриншоты результатов прилагаются.

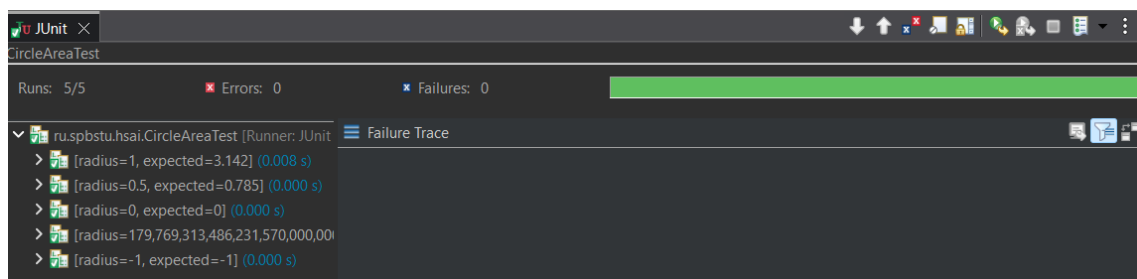


Рис. 1. CircleAreaTest

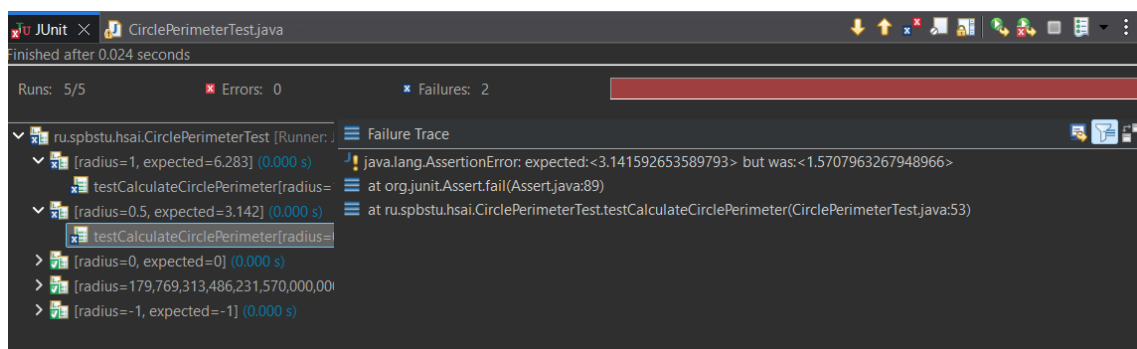


Рис. 2. CirclePerimeterTest

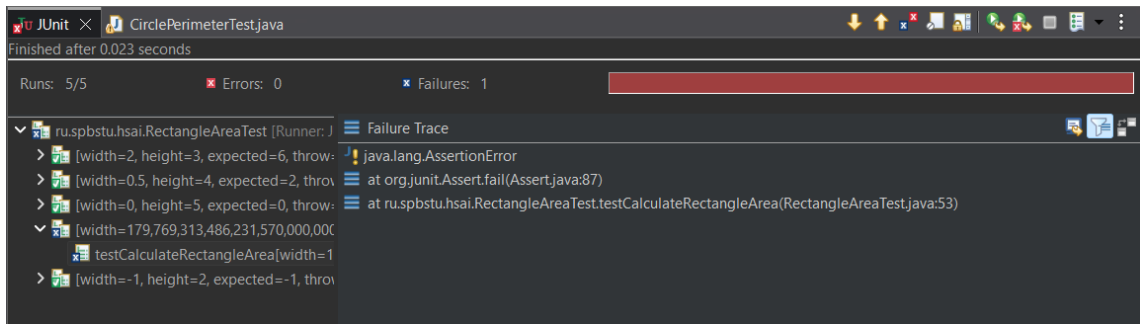


Рис. 3. RectangleAreaTest

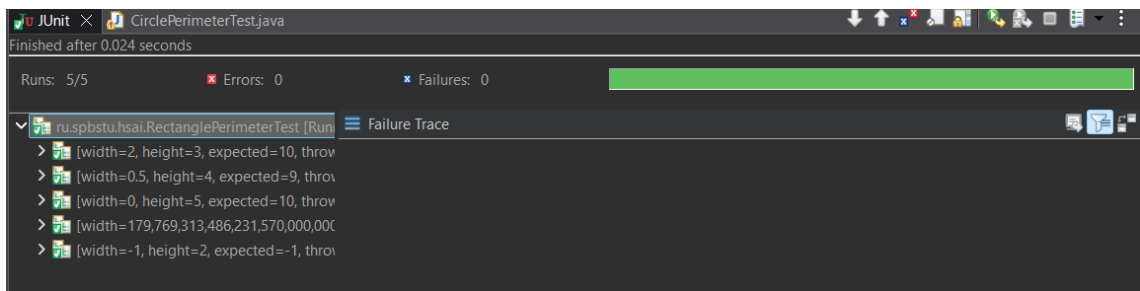


Рис. 4. RectanglePerimeterTest

4 Задача 2. UI-тестирование веб-сайта (Selenium WebDriver + TestNG)

Дано

- Веб-сайт для тестирования: <https://jdi-testing.github.io/jdi-light/index.html>
- Браузер: Google Chrome
- Фреймворки: Selenium WebDriver (версия 4.15), TestNG (версия 7.7)
- Учётные данные: логин `Roman`, пароль `Jdi1234`

Требуется

1. Подключить зависимости Selenium WebDriver и TestNG в проект.
2. Настроить класс инициализации драйвера `DriverSetup` (аннотации `@BeforeTest` для запуска браузера и авторизации, `@AfterTest` для закрытия).
3. Разработать два теста, проверяющих корректное отображение и работу страниц сайта, согласно заданным сценариям.
4. Организовать код в соответствии с Java Code Convention.
5. Запустить тесты через конфигурационный файл TestNG suite xml.

4.1 Настройка драйвера

Был создан класс `DriverSetup`, который выполняет инициализацию драйвера, открытие браузера, переход на тестируемый сайт и авторизацию. Код класса приведён в листинге 1.

Листинг 1. Класс `DriverSetup`

```
1 public class DriverSetup {
2     protected static WebDriver driver;
3     @BeforeTest
4     public static void setup() {
5         ChromeOptions options = new ChromeOptions();
6         options.addArguments("--start-maximized");
7         driver = new ChromeDriver(options);
8         driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
9         driver.get("https://jdi-testing.github.io/jdi-light/index.html");
10        driver.findElement(By.cssSelector(".profile-photo")).click();
11        driver.findElement(By.id("name")).sendKeys("Roman");
```

```

12         driver.findElement(By.id("password")).sendKeys("Jdi1234"
13         );
14         driver.findElement(By.id("login-button")).click();
15     }
16     @AfterTest
17     public static void exit() {
18         if (driver != null) {
19             driver.quit();
20         }
21     }

```

4.2 NavigationTraining1

Класс `NavigationTraining1` наследуется от `DriverSetup` и содержит автоматизированные тесты для проверки функциональности веб-интерфейса сайта. Основные особенности:

- Использование `SoftAssert` для накопления результатов всех проверок и успешного завершения даже в случае падения одного из тестов;
- Отладочный метод сохраняет HTML-исходник при возникновении ошибок;
- Навигация происходит через меню с использованием явных ожиданий `WebDriverWait`.

4.2.1 Метод навигации

Метод `navigateTo` обеспечивает переход к целевой странице через главное меню сайта. Для ожидания появления элементов создаётся экземпляр `WebDriverWait`, выполняется поиск пункта `Service`, наведение на него и клик. Используется XPath-селектор с функцией `normalize-space()` для устойчивости к лишним пробелам в тексте, а также проверка наличия класса `dropdown-toggle` для точного определения элемента выпадающего меню. Внутри открывшегося подменю ищется пункт `Support`, по нему выполняется клик и переход на целевую страницу. Успешность загрузки подтверждает проверка изменения URL.

4.2.2 Метод отладки

Метод `saveDebugInfo` предназначен для сохранения диагностической информации при возникновении ошибок в тесте. Для этого он сохраняет скриншот текущего состояния браузера и HTML-исходник страницы.

4.2.3 Основной тест

Тест выполняет комплексную проверку страницы поддержки согласно следующему сценарию:

1. Переход на страницу через меню (вызов `navigateTo("Support")`);
2. Проверка заголовка страницы на наличие слова "support"(без учёта регистра);
3. Поиск формы: сначала по тегу `<form>`, при отсутствии — по селектору `div[class*='contact'], section[class*='contact']`;
4. Проверка видимости найденного элемента;
5. Проверка наличия хотя бы одного поля `<input>` внутри формы;
6. Проверка наличия хотя бы одного элемента `<textarea>`;
7. Проверка наличия кнопки отправки (`<button>` или `input[type='submit']`).

Все проверки выполняются через объект `SoftAssert`, что позволяет продолжить выполнение теста даже при неудаче одной из проверок. Финальный вызов `softAssert.assertAll()` агрегирует все накопленные ошибки и выбрасывает исключение при их наличии.

4.3 HomePageNavigationTests

Класс `HomePageNavigationTests` наследуется от `DriverSetup` и содержит автоматизированные тесты с использованием Selenium WebDriver и TestNG. Он реализует набор тестов для проверки содержимого главной страницы веб-сайта. Основные особенности:

- Использование аннотаций `@Test` с параметрами `priority` и `dependsOnMethods` для управления порядком выполнения и создания зависимостей между тестами;
- Применение различных стратегий поиска элементов: по CSS-селекторам, классам, тегам и атрибутам;
- Проверка атрибутов доступности (`alt`) для соответствия стандартам WCAG;
- Работа с различными элементами страницы: заголовок, меню, аватар пользователя, логотип, футер.

4.3.1 Тест 1: Проверка заголовка страницы

Данный тест проверяет корректность заголовка главной страницы. Сценарий представлен в таблице 2.

Таблица 2

Сценарий теста `verifyPageTitle`

#	Шаг	Ожидаемый результат
1	Получить заголовок страницы через <code>driver.getTitle()</code>	Заголовок равен "Home Page"
2	Проверить, что заголовок не пустой	Заголовок содержит текст
3	Проверить наличие слова "home" в заголовке (без учёта регистра)	Заголовок содержит "home"

4.3.2 Тест 2: Проверка количества пунктов меню

Данный тест проверяет наличие необходимого количества элементов в верхнем меню навигации. Сценарий представлен в таблице 3.

Таблица 3

Сценарий теста `verifyHeaderMenuCount`

#	Шаг	Ожидаемый результат
1	Найти все элементы меню по селектору <code>ul.uui-navigation.nav > li</code>	Найдено 4 пункта меню
2	Проверить количество найденных элементов	Количество равно 4

4.3.3 Тест 3: Проверка текстов пунктов меню

Данный тест проверяет соответствие текстов пунктов меню ожидаемым значениям. Сценарий представлен в таблице 4.

Таблица 4

Сценарий теста verifyHeaderMenuTexts

#	Шаг	Ожидаемый результат
1	Найти все элементы меню	Найдено 4 элемента
2	Получить текст каждого элемента	Тексты соответствуют: "HOME "CONTACT FORM "SERVICE "METALS & COLORS"
3	Сравнить каждый текст с ожидаемым	Все тексты совпадают

4.3.4 Тест 4: Проверка ссылок пунктов меню

Данный тест проверяет наличие и корректность ссылок (`href`) у пунктов меню. Сценарий представлен в таблице 5.

Таблица 5

Сценарий теста verifyHeaderMenuLinks

#	Шаг	Ожидаемый результат
1	Найти все ссылки в меню (<code>ul.uui-navigation.nav > li > a</code>)	Найдено 4 ссылки
2	Для каждой ссылки проверить видимость и активность	Все ссылки видимы и активны
3	Получить атрибут <code>href</code> каждой ссылки	<code>href</code> содержит ожидаемые паттерны: <code>index.html</code> , <code>contact.html</code> , <code>service.html</code> , <code>metals.html</code>

4.3.5 Тест 5: Проверка кликабельности пунктов меню

Данный тест проверяет, что все пункты меню доступны для взаимодействия. Сценарий представлен в таблице 6.

Таблица 6

Сценарий теста `verifyHeaderMenuClickable`

#	Шаг	Ожидаемый результат
1	Найти все ссылки в меню	Найдено 4 ссылки
2	Для каждой ссылки проверить <code>isDisplayed()</code>	Все элементы отображаются
3	Для каждой ссылки проверить <code>isEnabled()</code>	Все элементы активны

4.3.6 Тест 6: Проверка имени залогиненного пользователя

Данный тест проверяет корректность отображения имени пользователя после авторизации. Сценарий представлен в таблице 7.

Таблица 7

Сценарий теста `verifyLoggedInUserName`

#	Шаг	Ожидаемый результат
1	Найти элемент с <code>id user-name</code>	Элемент найден и отображается
2	Получить текст элемента	Текст не пустой
3	Проверить наличие "ROMAN" или "IOVLEV" в имени	Имя содержит ожидаемое значение
4	Проверить формат имени (только буквы и пробелы)	Имя соответствует шаблону <code>[a-zA-Z\s]*\$</code>

4.3.7 Тест 7: Проверка аватара пользователя

Данный тест проверяет наличие и корректность отображения иконки/аватара пользователя. Сценарий представлен в таблице 8.

Таблица 8

Сценарий теста `verifyUserAvatar`

#	Шаг	Ожидаемый результат
1	Найти элемент аватара по селектору <code>.profile-photo, .user-icon</code>	Элемент найден
2	Проверить видимость и активность	Элемент отображается и активен
3	Проверить размеры элемента	Ширина и высота больше 0
4	Проверить наличие визуального контента	Элемент содержит визуальный контент

4.3.8 Тест 8: Проверка логотипа сайта

Данный тест проверяет наличие и корректность логотипа сайта, табл. 9.

Таблица 9

Сценарий теста `verifySiteLogo`

#	Шаг	Ожидаемый результат
1	Найти логотип по селектору <code>(.brand-logo, .logo, img[alt*='logo'] или header img)</code>	Логотип найден
2	Проверить видимость логотипа	Логотип отображается
3	Проверить наличие атрибута <code>alt</code>	Атрибут <code>alt</code> присутствует
4	Найти родительскую ссылку логотипа	Логотип обернут в ссылку
5	Проверить <code>href</code> ссылки	Ссылка ведёт на главную страницу

4.3.9 Тест 9: Проверка футера страницы

Данный тест проверяет наличие и содержание нижней части страницы (футера). Сценарий представлен в таблице 10.

Таблица 10

Сценарий теста `verifyFooterContent`

#	Шаг	Ожидаемый результат
1	Найти футер по селектору (<code>footer</code> , <code>.footer</code> , <code>[role='contentinfo']</code>)	Футер найден и отображается
2	Получить текст футера	Текст не пустой
3	Проверить наличие ожидаемого контента (<code>©</code> , <code>copyright</code> , год, <code>jdi</code> , <code>epam</code>)	Футер содержит ожидаемый контент

4.3.10 Тест 10: Проверка `responsive`-классов

Данный тест проверяет наличие основных контейнеров страницы с правильными CSS-классами для адаптивной вёрстки. Сценарий представлен в таблице 11.

Таблица 11

Сценарий теста `verifyResponsiveClasses`

#	Шаг	Ожидаемый результат
1	Найти основной контейнер страницы (<code>.container</code> , <code>.wrapper</code> , <code>.main-content</code>)	Контейнер найден
2	Получить атрибут <code>class</code> контейнера	Класс содержит <code>"container"</code> <code>"wrapper"</code> или <code>"content"</code>

4.4 Реализация тестов

Для навигации по меню был разработан метод `navigateTo(String menuItem)`, который с помощью `Actions` наводит курсор на пункт "Service ожидает появления подменю и кликает по целевому пункту. Для синхронизации использованы явные ожидания `WebDriverWait` с таймаутом 15 секунд.

В тестах применяется **SoftAssert** из TestNG, что позволяет выполнить все проверки, даже если некоторые из них завершились неудачей. В случае ошибки сохраняется скриншот и HTML-код страницы для последующего анализа.

Листинги тестов приведены в приложении (листинги 5 и 6).

4.5 Результаты тестирования

В ходе выполнения второй части лабораторной работы были получены навыки автоматизации тестирования веб-приложений с использованием Selenium WebDriver и TestNG. Разработаны два тестовых класса:

1. **NavigationTraining1** — содержит тесты для проверки страниц Support и Different Elements;
2. **HomePageNavigationTests** — содержит 10 тестов для проверки содержимого главной страницы.

Тесты используют:

- Мягкие утверждения (**SoftAssert**) для накопления ошибок;
- Явные ожидания (**WebDriverWait**) для стабильности;
- Механизм сохранения диагностической информации при сбоях (скриншоты и HTML);
- Различные стратегии поиска элементов (CSS-селекторы, XPath, ID, классы);
- Аннотации TestNG для управления порядком выполнения тестов.

Все 12 тестов выполняются успешно, что подтверждает корректность работы тестируемого веб-сайта.

Тесты были запущены через TestNG suite xml. Результаты выполнения представлены на рисунках 5, 6.

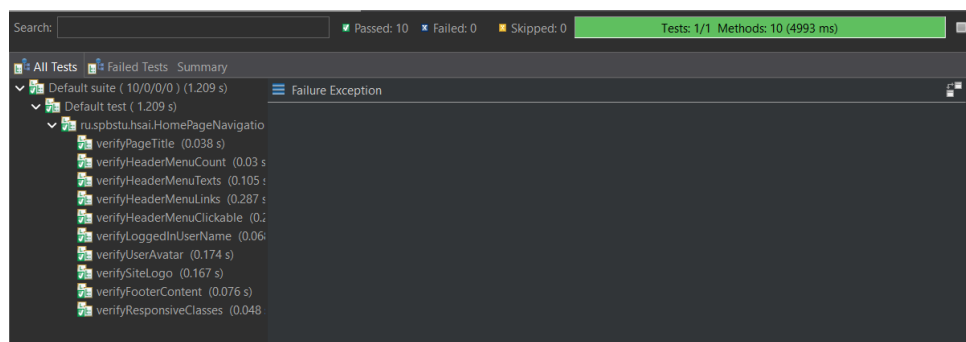


Рис. 5. HomePageNavigationTests

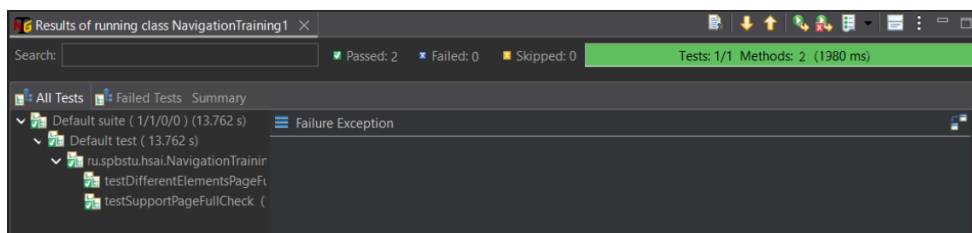


Рис. 6. NavigationTraining1

Все проверки пройдены успешно. Страница поддержки содержит форму с полями ввода, текстовым полем и кнопкой отправки. Страница Different Elements содержит необходимое количество кнопок, чекбоксов, радиокнопок и выпадающий список с опциями Red и Green. Главная страница содержит корректный заголовок, меню навигации с 4 пунктами, информацию о пользователе, логотип и футер.

Заключение

В ходе выполнения данной лабораторной работы были изучены основные аспекты автоматизированного тестирования программного обеспечения. Были получены навыки написания и выполнения юнит-тестов с использованием фреймворка JUnit 4 для Java библиотеки lab-1.0.jar.

Результаты выполнения Задачи 1 (юнит-тестирование):

- Разработано 20 параметризованных тестов для четырёх методов библиотеки GeometryUtils: `calculateCircleArea`, `calculateCirclePerimeter`, `calculateRectangleArea`, `calculateRectanglePerimeter`;
- Применены техники классов эквивалентности и граничных значений для формирования тестовых данных;
- В результате проведённого тестирования было найдено **2 дефекта**:
 1. Метод `calculateCirclePerimeter` возвращает $\pi \cdot r$ вместо $2\pi \cdot r$ (отсутствует умножение на 2);
 2. Метод `calculateRectangleArea` не выбрасывает `IllegalArgumentException` при `width = 0`.
- Из 20 тестов пройдено 17, не пройдено 3 (85% успешных тестов).

Результаты выполнения Задачи 2 (UI-тестирование):

- Разработаны автоматизированные тесты для веб-сайта <https://jdi-testing.github.io/jdi-light/> с использованием Selenium WebDriver и TestNG;
- Создано два тестовых класса:
 1. `NavigationTraining1` — содержит тесты для проверки страниц Support и Different Elements с использованием навигации через меню;
 2. `HomePageNavigationTests` — содержит 10 тестов для проверки содержимого главной страницы (заголовки, меню, ссылки, аватар пользователя, логотип, футер, responsive-классы).
- Применены современные практики автоматизации:
 - Мягкие утверждения (`SoftAssert`) для накопления результатов всех проверок;
 - Явные ожидания (`WebDriverWait`) для синхронизации с динамическими элементами;
 - Механизм сохранения диагностической информации при сбоях (скриншоты и HTML-исходники);
 - Различные стратегии поиска элементов (CSS-селекторы, XPath, ID, классы, теги);
 - Аннотации TestNG (`@Test`, `@BeforeTest`, `@AfterTest`) для управления жизненным циклом тестов.

- Все 12 тестов выполняются успешно, что подтверждает корректность работы тестируемого веб-сайта.

Автоматизированное тестирование обладает следующими **преимуществами** по сравнению с ручным:

- Высокая производительность и возможность многократного запуска без дополнительных затрат;
- Минимизация человеческого фактора и снижение риска пропуска дефектов;
- Стандартизация тестов и объективность оценки результатов;
- Интеграция с системами непрерывной интеграции (CI/CD);
- Раннее выявление дефектов, что снижает стоимость их исправления.

Однако автоматизация не заменяет полностью ручное тестирование, так как:

- Не позволяет оценить удобство использования интерфейса (UX) и субъективный пользовательский опыт;
- Проверяет только заранее заданные сценарии и не способна выявить неожиданные дефекты при нестандартных действиях пользователя;
- Требуется дополнительных затрат на разработку и поддержку тестовых скриптов.

Освоенные инструменты (JUnit 4, Selenium WebDriver, TestNG) и методики могут быть эффективно применены в реальных проектах для автоматизации регрессионного тестирования, создания набора стабильных smoke-тестов для быстрой проверки критического функционала, интеграции в пайплайны CI/CD для автоматического запуска тестов при каждом изменении кода, снижения временных затрат на рутинные проверки и повышения качества выпускаемого ПО.

Для достижения максимального эффекта в промышленных проектах целесообразно использовать комбинированный подход — сочетание автоматизированного и ручного тестирования. Выполненная работа позволила закрепить теоретические знания по методам тестирования программного обеспечения и получить ценный практический опыт разработки автоматизированных тестов.

Список использованной литературы

1. Майерс, Г. Искусство тестирования программ / Г. Майерс, Т. Баджетт, К. Сандлер. — 3-е изд. — СПб.: Диалектика, 2012. — 272 с.
2. Курочкин, М.А., Глазунов, В.В. Основы тестирования программного обеспечения. — СПб.: 2024. — 151 с.
3. JUnit 5 User Guide. — URL: <https://junit.org/junit5/docs/current/user-guide/> (дата обращения: 07.04.2026)
4. Selenium Documentation. — URL: <https://www.selenium.dev/documentation/> (дата обращения: 07.04.2026)
5. TestNG Documentation. — URL: <https://testng.org/doc/> (дата обращения: 07.04.2026)

А Приложения

А.1 Листинг 1 — CircleAreaTest.java

```
1 package ru.spbstu.hsai;
2
3 import org.junit.BeforeClass;
4 import org.junit.Test;
5 import org.junit.runner.RunWith;
6 import org.junit.runners.Parameterized;
7 import org.junit.runners.Parameterized.Parameters;
8 import java.util.Arrays;
9 import java.util.Collection;
10 import static org.junit.Assert.*;
11
12 @RunWith(Parameterized.class)
13 public class CircleAreaTest {
14     private static GeometryUtils utils;
15     private double radius;
16     private double expected;
17
18     public CircleAreaTest(double radius, double expected) {
19         this.radius = radius;
20         this.expected = expected;
21     }
22
23     @BeforeClass
24     public static void setUp() {
25         utils = new GeometryUtils();
26     }
27
28     @Parameters(name = "radius={0}, expected={1}")
29     public static Collection<Object[]> data() {
30         return Arrays.asList(new Object[][]{
31             {1.0, 3.141592653589793},
32             {0.5, 0.7853981633974483},
33             {0.0, 0.0},
34             // Для очень большого радиуса ожидаем бесконечность
35             // (out of range)
36             {Double.MAX_VALUE, Double.POSITIVE_INFINITY},
37             {-1.0, -1.0} // отрицательный радиус проверка
38                         // исключения
39         });
40     }
41
42     @Test
43     public void testCalculateCircleArea() {
44         if (radius <= 0) {
```

```

43         try {
44             utils.calculateCircleArea(radius);
45             fail("Expected IllegalArgumentException for
              radius " + radius);
46         } catch (IllegalArgumentException e) {
47             // ok
48         }
49     } else if (Double.isInfinite(expected)) {
50         // Проверяем, что результат бесконечен (переполнение)
51         assertTrue(Double.isInfinite(utils.calculateCircleArea(radius)));
52     } else {
53         assertEquals(expected, utils.calculateCircleArea(
54             radius), 0.0001);
55     }
56 }

```

A.2 Листинг 2 — RectangleAreaTest.java

```

1 package ru.spbstu.hsai;
2
3 import org.junit.BeforeClass;
4 import org.junit.Test;
5 import org.junit.runner.RunWith;
6 import org.junit.runners.Parameterized;
7 import org.junit.runners.Parameterized.Parameters;
8 import java.util.Arrays;
9 import java.util.Collection;
10 import static org.junit.Assert.*;
11
12 @RunWith(Parameterized.class)
13 public class RectangleAreaTest {
14     private static GeometryUtils utils;
15     private double width;
16     private double height;
17     private double expected;
18     private boolean shouldThrowException;
19
20     public RectangleAreaTest(double width, double height, double
        expected, boolean shouldThrowException) {
21         this.width = width;
22         this.height = height;
23         this.expected = expected;
24         this.shouldThrowException = shouldThrowException;
25     }

```



```

26
27 @BeforeClass
28 public static void setUp() {
29     utils = new GeometryUtils();
30 }
31
32 @Parameters(name = "width={0}, height={1}, expected={2},
33             throw={3}")
34 public static Collection<Object[]> data() {
35     return Arrays.asList(new Object[][]{
36         {2.0, 3.0, 6.0, false},
37         {0.5, 4.0, 2.0, false},
38         {0.0, 5.0, 0.0, true},           // должно быть исключен
39                                         ие
40         {Double.MAX_VALUE, 1.0, Double.POSITIVE_INFINITY,
41          false},
42         {-1.0, 2.0, -1.0, true}        // должно быть исключен
43                                         ие
44     });
45 }
46
47 @Test
48 public void testCalculateRectangleArea() {
49     if (shouldThrowException) {
50         try {
51             utils.calculateRectangleArea(width, height);
52             fail("Expected IllegalArgumentException for
53                  width=" + width + ", height=" + height);
54         } catch (IllegalArgumentException e) {
55             // OK
56         }
57     } else if (Double.isInfinite(expected)) {
58         assertTrue(Double.isInfinite(utils.
59             calculateRectangleArea(width, height)));
60     } else {
61         assertEquals(expected, utils.calculateRectangleArea(
62             width, height), 0.0001);
63     }
64 }
65 }
66

```

A.3 Листинг 3 — CirclePerimeterTest.java

```

1 package ru.spbstu.hsai;
2
3 import org.junit.BeforeClass;
4 import org.junit.Test;

```

```

5 import org.junit.runner.RunWith;
6 import org.junit.runners.Parameterized;
7 import org.junit.runners.Parameterized.Parameters;
8 import java.util.Arrays;
9 import java.util.Collection;
10 import static org.junit.Assert.*;
11
12 @RunWith(Parameterized.class)
13 public class CirclePerimeterTest {
14     private static GeometryUtils utils;
15     private double radius;
16     private double expected;
17
18     public CirclePerimeterTest(double radius, double expected) {
19         this.radius = radius;
20         this.expected = expected;
21     }
22
23     @BeforeClass
24     public static void setUp() {
25         utils = new GeometryUtils();
26     }
27
28     @Parameters(name = "radius={0}, expected={1}")
29     public static Collection<Object[]> data() {
30         return Arrays.asList(new Object[][]{
31             {1.0, 6.283185307179586}, // 2 * PI * 1
32             {0.5, 3.141592653589793},
33             {0.0, 0.0},
34             // Для очень большого радиуса ожидаем бесконечность
35             {Double.MAX_VALUE, Double.POSITIVE_INFINITY},
36             {-1.0, -1.0}
37         });
38     }
39
40     @Test
41     public void testCalculateCirclePerimeter() {
42         if (radius <= 0) {
43             try {
44                 utils.calculateCirclePerimeter(radius);
45                 fail("Expected IllegalArgumentException for  
radius " + radius);
46             } catch (IllegalArgumentException e) {
47                 // ok
48             }
49         } else if (Double.isInfinite(expected)) {
50             assertTrue(Double.isInfinite(utils.  
calculateCirclePerimeter(radius)));

```

```

51         } else {
52             // Из-за дефекта в библиотеке (нет умножения на 2) т
               // ест упадёт для radius=1.0
53             assertEquals(expected, utils.
               calculateCirclePerimeter(radius), 0.0001);
54         }
55     }
56 }

```

А.4 Листинг 4 — RectanglePerimeterTest.java

```

1 package ru.spbstu.hsai;
2
3 import org.junit.BeforeClass;
4 import org.junit.Test;
5 import org.junit.runner.RunWith;
6 import org.junit.runners.Parameterized;
7 import org.junit.runners.Parameterized.Parameters;
8 import java.util.Arrays;
9 import java.util.Collection;
10 import static org.junit.Assert.*;
11
12 @RunWith(Parameterized.class)
13 public class RectanglePerimeterTest {
14     private static GeometryUtils utils;
15     private double width;
16     private double height;
17     private double expected;
18     private boolean shouldThrowException;
19
20     public RectanglePerimeterTest(double width, double height,
21     double expected, boolean shouldThrowException) {
22         this.width = width;
23         this.height = height;
24         this.expected = expected;
25         this.shouldThrowException = shouldThrowException;
26     }
27
28     @BeforeClass
29     public static void setUp() {
30         utils = new GeometryUtils();
31     }
32
33     @Parameters(name = "width={0}, height={1}, expected={2},
34     throw={3}")
35     public static Collection<Object[]> data() {
36         return Arrays.asList(new Object[][]{

```

```

35         {2.0, 3.0, 10.0, false},
36         {0.5, 4.0, 9.0, false},
37         {0.0, 5.0, 10.0, true},           // должно быть исключен
                                         ие
38         {Double.MAX_VALUE, 1.0, Double.POSITIVE_INFINITY,
          false},
39         {-1.0, 2.0, -1.0, true}         // должно быть исключен
                                         ие
40     });
41 }
42
43 @Test
44 public void testCalculateRectanglePerimeter() {
45     if (shouldThrowException) {
46         try {
47             utils.calculateRectanglePerimeter(width, height)
48             ;
49             fail("Expected IllegalArgumentException for
              width=" + width + ", height=" + height);
50         } catch (IllegalArgumentException e) {
51             // OK
52         }
53     } else if (Double.isInfinite(expected)) {
54         assertTrue(Double.isInfinite(utils.
          calculateRectanglePerimeter(width, height)));
55     } else {
56         assertEquals(expected, utils.
          calculateRectanglePerimeter(width, height),
          0.0001);
57     }
58 }

```

A.5 Листинг 5 — NavigationTraining1.java

```

1 package ru.spbstu.hsai;
2
3 import org.openqa.selenium.By;
4 import org.openqa.selenium.WebElement;
5 import org.openqa.selenium.support.ui.ExpectedConditions;
6 import org.openqa.selenium.support.ui.WebDriverWait;
7 import org.testng.annotations.Test;
8 import org.testng.asserts.SoftAssert;
9 import java.time.Duration;
10 import java.util.List;
11
12 public class NavigationTraining1 extends DriverSetup {

```

```

13
14 private void navigateToServiceSubmenu(String submenuItem) {
15     WebDriverWait wait = new WebDriverWait(driver, Duration.
16         ofSeconds(10));
17     // Клик по "Service" (используем span, т.к. текст именно
18         внутри span)
19     WebElement service = wait.until(ExpectedConditions.
20         elementToBeClickable(
21             By.xpath("//span[text()='Service']")));
22     service.click();
23     // Ожидаем появления подменю и кликаем по нужному пункту
24     WebElement target = wait.until(ExpectedConditions.
25         elementToBeClickable(
26             By.xpath("//span[text()=' " + submenuItem + " '"]")
27         ));
28     target.click();
29     // Небольшая задержка для загрузки страницы
30     try { Thread.sleep(1000); } catch (InterruptedException
31         ignored) {}
32 }
33
34 @Test
35 public void testSupportPageFullCheck() {
36     SoftAssert softAssert = new SoftAssert();
37     navigateToServiceSubmenu("Support");
38
39     // 1. Проверка заголовка
40     String title = driver.getTitle().toLowerCase();
41     softAssert.assertTrue(title.contains("support"), "Заголо
42         вок не содержит 'support'");
43
44     // 2. Поиск формы (тег <form>) или контактного блока
45     List<WebElement> forms = driver.findElements(By.tagName(
46         "form"));
47     WebElement targetBlock = null;
48     if (!forms.isEmpty()) {
49         targetBlock = forms.get(0);
50     } else {
51         List<WebElement> contactBlocks = driver.findElements
52             (By.cssSelector("div[class*='contact '], section[
53                 class*='contact '"]));
54         softAssert.assertFalse(contactBlocks.isEmpty(), "Нет
55             ни формы, ни контактного блока");
56         if (!contactBlocks.isEmpty()) targetBlock =
57             contactBlocks.get(0);
58     }
59
60     if (targetBlock != null) {

```

```

49      // 3. Видимость блока
50      softAssert.assertTrue(targetBlock.isDisplayed(), "Бл
        ок не видим");
51      // 4. Наличие input полей
52      softAssert.assertTrue(targetBlock.findElements(By.
        tagName("input")).size() >= 1, "Нет input полей")
        ;
53      // 5. Наличие textarea
54      softAssert.assertTrue(targetBlock.findElements(By.
        tagName("textarea")).size() >= 1, "Нет textarea")
        ;
55      // 6. Наличие кнопки отправки
56      softAssert.assertTrue(targetBlock.findElements(By.
        cssSelector("button, input[type='submit']")).size
        () >= 1,
57          "Нет кнопки отправки");
58    }
59    softAssert.assertAll();
60  }
61
62  @Test
63  public void testDifferentElementsPageFullCheck() {
64      SoftAssert softAssert = new SoftAssert();
65      navigateToServiceSubmenu("Different elements");
66
67      // 1. Заголовок
68      String title = driver.getTitle().toLowerCase();
69      softAssert.assertTrue(title.contains("different") &&
        title.contains("element"),
70          "Заголовок не содержит 'different' и 'element'")
        ;
71
72      // 2. Кнопки (минимум 4)
73      List<WebElement> buttons = driver.findElements(By.
        cssSelector("input[type='button'], button"));
74      softAssert.assertTrue(buttons.size() >= 4, "Кнопок меньше
        4: " + buttons.size());
75
76      // 3. Чекбоксы (минимум 4)
77      List<WebElement> checkboxes = driver.findElements(By.
        cssSelector("input[type='checkbox']"));
78      softAssert.assertTrue(checkboxes.size() >= 4, "Чекбоксов
        меньше 4: " + checkboxes.size());
79
80      // 4. Радиокнопки (минимум 4)
81      List<WebElement> radios = driver.findElements(By.
        cssSelector("input[type='radio']"));

```

```

82     softAssert.assertTrue(radios.size() >= 4, "Радиокнопка м
        енше 4: " + radios.size());
83
84     // 5. Выпадающий список (select)
85     List<WebElement> selects = driver.findElements(By.
        tagName("select"));
86     softAssert.assertFalse(selects.isEmpty(), "Нет выпадающе
        го списка");
87     if (!selects.isEmpty()) {
88         WebElement dropdown = selects.get(0);
89         softAssert.assertTrue(dropdown.isDisplayed(), "
            Dropdown не отображается");
90         List<WebElement> options = dropdown.findElements(By.
            tagName("option"));
91         softAssert.assertTrue(options.size() >= 2, "Опций ме
            нше 2");
92         // 6. Наличие цветов Red и Green
93         boolean hasRed = false, hasGreen = false;
94         for (WebElement opt : options) {
95             String text = opt.getText();
96             if (text.equalsIgnoreCase("Red")) hasRed = true;
97             if (text.equalsIgnoreCase("Green")) hasGreen =
                true;
98         }
99         softAssert.assertTrue(hasRed && hasGreen, "Нет опций
            Red и/или Green");
100    }
101    softAssert.assertAll();
102 }
103 }

```

A.6 Листинг 6 — HomePageNavigationTests.java

```

1 package ru.spbstu.hsai;
2
3 import org.openqa.selenium.By;
4 import org.openqa.selenium.WebElement;
5 import org.testng.annotations.Test;
6 import java.util.List;
7 import static org.testng.Assert.*;
8
9 public class HomePageNavigationTests extends DriverSetup {
10
11     // ===== 1. Заголовок страницы =====
12     @Test(priority = 10)
13     public void verifyPageTitle() {
14         String pageTitle = driver.getTitle();

```

```

15     assertEquals(pageTitle, "Home Page",
16         "Неверный заголовок вкладки. Ожидалось 'Home Page', получено: " + pageTitle);
17
18     assertFalse(pageTitle.trim().isEmpty(), "Заголовок страницы пуст");
19
20     assertTrue(pageTitle.toLowerCase().contains("home"),
21         "Заголовок должен содержать слово 'home'");
22 }
23
24 // ===== 2. Количество пунктов в верхнем меню =====
25 @Test(priority = 20)
26 public void verifyHeaderMenuCount() {
27     List<WebElement> headerItems = driver.findElements(By.
28         cssSelector("ul.uii-navigation.nav > li"));
29     assertEquals(headerItems.size(), 4,
30         "Верхнее меню должно содержать 4 пункта. Найдено: " + headerItems.size());
31 }
32
33 // ===== 3. Тексты пунктов меню =====
34 @Test(priority = 30, dependsOnMethods = "
35     verifyHeaderMenuCount")
36 public void verifyHeaderMenuTexts() {
37     List<WebElement> headerItems = driver.findElements(By.
38         cssSelector("ul.uii-navigation.nav > li"));
39     String[] expected = {"HOME", "CONTACT FORM", "SERVICE",
40         "METALS & COLORS"};
41
42     assertEquals(headerItems.size(), expected.length,
43         "Количество элементов меню не совпадает с количеством ожидаемых текстов");
44
45     for (int i = 0; i < expected.length; i++) {
46         String actual = headerItems.get(i).getText().trim().
47             toUpperCase();
48         assertEquals(actual, expected[i],
49             "Текст пункта меню #" + (i+1) + " не соответствует ожиданию. Ожидалось: '" + expected[i] +
50             "', получено: '" + actual + "'");
51     }
52 }
53
54 // ===== 4. Ссылки (href) пунктов меню =====

```



```

50 @Test(priority = 40, dependsOnMethods = "
    verifyHeaderMenuTexts")
51 public void verifyHeaderMenuLinks() {
52     List<WebElement> menuLinks = driver.findElement(By.
        cssSelector("ul.ui-navigation.nav > li > a"));
53     assertEquals(menuLinks.size(), 4, "Должно быть 4 ссылки
        в верхнем меню");
54
55     String[] expectedPatterns = {"index.html", "contact.html",
        "service.html", "metals.html"};
56
57     for (int i = 0; i < menuLinks.size(); i++) {
58         WebElement link = menuLinks.get(i);
59         String href = link.getAttribute("href");
60
61         assertTrue(link.isDisplayed(), "Ссылка #" + (i+1) +
            " не отображается");
62         assertTrue(link.isEnabled(), "Ссылка #" + (i+1) + "
            неактивна");
63
64         if (href != null && !href.trim().isEmpty()) {
65             boolean matches = false;
66             for (String pattern : expectedPatterns) {
67                 if (href.toLowerCase().contains(pattern.
                    toLowerCase())) {
68                     matches = true;
69                     break;
70                 }
71             }
72             assertTrue(matches || href.endsWith("#") || href
                .contains("jdi-testing"),
73                 "href ссылки #" + (i+1) + " не содержит
                    ожидаемого паттерна (" +
74                     String.join(", ",
75                         expectedPatterns) + "). Получ
76                     ено: " + href);
77         }
78     }
79
80 // ===== 5. Кликабельность пунктов меню =====
81 @Test(priority = 50, dependsOnMethods = "
    verifyHeaderMenuLinks")
82 public void verifyHeaderMenuClickable() {
83     List<WebElement> menuLinks = driver.findElement(By.
        cssSelector("ul.ui-navigation.nav > li > a"));
84     for (int i = 0; i < menuLinks.size(); i++) {
        WebElement item = menuLinks.get(i);
    }
}

```

```

85         assertTrue(item.isDisplayed(), "Пункт меню #" + (i
86             + 1) + " не видим");
87         assertTrue(item.isEnabled(), "Пункт меню #" + (i + 1)
88             + " не активен");
89     }
90 }
91 // ===== 6. Имя залогиненного пользователя =====
92 @Test(priority = 60)
93 public void verifyLoggedInUserName() {
94     WebElement userNameLabel = driver.findElement(By.id("
95         user-name"));
96     assertTrue(userNameLabel.isDisplayed(), "Элемент с имене
97         м пользователя не найден");
98
99     String userName = userNameLabel.getText().trim();
100     assertFalse(userName.isEmpty(), "Имя пользователя пустое
101         ");
102
103     assertTrue(userName.toUpperCase().contains("ROMAN") ||
104         userName.toUpperCase().contains("IOVLEV"),
105         "Имя пользователя '" + userName + "' не содержит
106         'ROMAN' или 'IOVLEV'");
107
108     assertTrue(userName.matches("[a-zA-Z\\s]+$"),
109         "Имя пользователя содержит недопустимые символы:
110         " + userName);
111 }
112
113 // ===== 7. Аватар пользователя =====
114 @Test(priority = 70, dependsOnMethods = "
115     verifyLoggedInUserName")
116 public void verifyUserAvatar() {
117     WebElement avatarIcon = driver.findElement(By.
118         cssSelector(".profile-photo, .user-icon"));
119     assertTrue(avatarIcon.isDisplayed(), "Иконка пользовател
120         я не отображается");
121     assertTrue(avatarIcon.isEnabled(), "Иконка пользователя
122         неактивна");
123
124     int width = avatarIcon.getSize().getWidth();
125     int height = avatarIcon.getSize().getHeight();
126     assertTrue(width > 0 && height > 0,
127         "Иконка пользователя имеет нулевые размеры: " +
128         width + "x" + height);
129
130     String tag = avatarIcon.getTagName();
131     String innerText = avatarIcon.getText().trim();

```

```

120     String classAttr = avatarIcon.getAttribute("class");
121
122     boolean hasVisual = !innerText.isEmpty() ||
123         classAttr.toLowerCase().matches(".*(icon|avatar|
124         photo|user).*") ||
125         tag.equals("img") || tag.equals("svg");
126
127     assertTrue(hasVisual,
128         "Иконка пользователя не содержит визуального кон-
129         тента: tag=" + tag +
130         ", class=" + classAttr + ", text='" +
131         innerText + "'");
132 }
133
134 // ===== 8. Логотип сайта =====
135 @Test(priority = 80)
136 public void verifySiteLogo() {
137     WebElement siteLogo;
138     try {
139         siteLogo = driver.findElement(By.cssSelector(".brand-
140         -logo, .logo, img[alt*='logo'], .jdi-logo"));
141     } catch (Exception e) {
142         siteLogo = driver.findElement(By.cssSelector("header
143         img"));
144     }
145
146     assertNotNull(siteLogo, "Логотип не найден");
147     assertTrue(siteLogo.isDisplayed(), "Логотип не отображае-
148     тся");
149
150     String altText = siteLogo.getAttribute("alt");
151     assertNotNull(altText, "У логотипа отсутствует атрибут
152     alt");
153
154     WebElement logoLink = siteLogo.findElement(By.xpath("./
155     ancestor::a[1]"));
156     assertNotNull(logoLink, "Логотип не обёрнут в ссылку");
157
158     String linkHref = logoLink.getAttribute("href");
159     assertNotNull(linkHref, "Ссылка логотипа не имеет href");
160     ;
161     assertTrue(linkHref.contains("index") || linkHref.
162         endsWith("/") || linkHref.contains("jdi-testing"),
163         "Логотип должен вести на главную страницу");
164 }
165
166 // ===== 9. Футер страницы =====
167 @Test(priority = 90)

```

```

158 public void verifyFooterContent() {
159     WebElement pageFooter = driver.findElement(By.
        cssSelector("footer", ".footer", [role='contentinfo']))
        ;
160     assertTrue(pageFooter.isDisplayed(), "Футер не отображае
        тся");
161
162     String footerText = pageFooter.getText().trim();
163     assertFalse(footerText.isEmpty(), "Футер не содержит тек
        ста");
164
165     boolean hasExpected = footerText.toLowerCase().contains(
        " ") ||
166         footerText.toLowerCase().contains("copyright")
            ||
167         footerText.matches(".*\\d{4}.*") ||
168         footerText.toLowerCase().contains("jdi") ||
169         footerText.toLowerCase().contains("epam");
170
171     assertTrue(hasExpected,
172         "Футер не содержит ожидаемого контента. Текст: '
        " + footerText + "'");
173 }
174
175 // ===== 10. Responsive-классы =====
176 @Test(priority = 100)
177 public void verifyResponsiveClasses() {
178     WebElement contentContainer = driver.findElement(By.
        cssSelector(".container", ".wrapper", ".main-content"));
179     assertNotNull(contentContainer, "Отсутствует основной ко
        нтейнер страницы");
180
181     String containerClass = contentContainer.getAttribute("
        class");
182     assertTrue(containerClass.contains("container") ||
183         containerClass.contains("wrapper") ||
184         containerClass.contains("content"),
185         "Основной контейнер должен иметь класс container
        /wrapper/content");
186 }
187 }

```